STM32片上外设

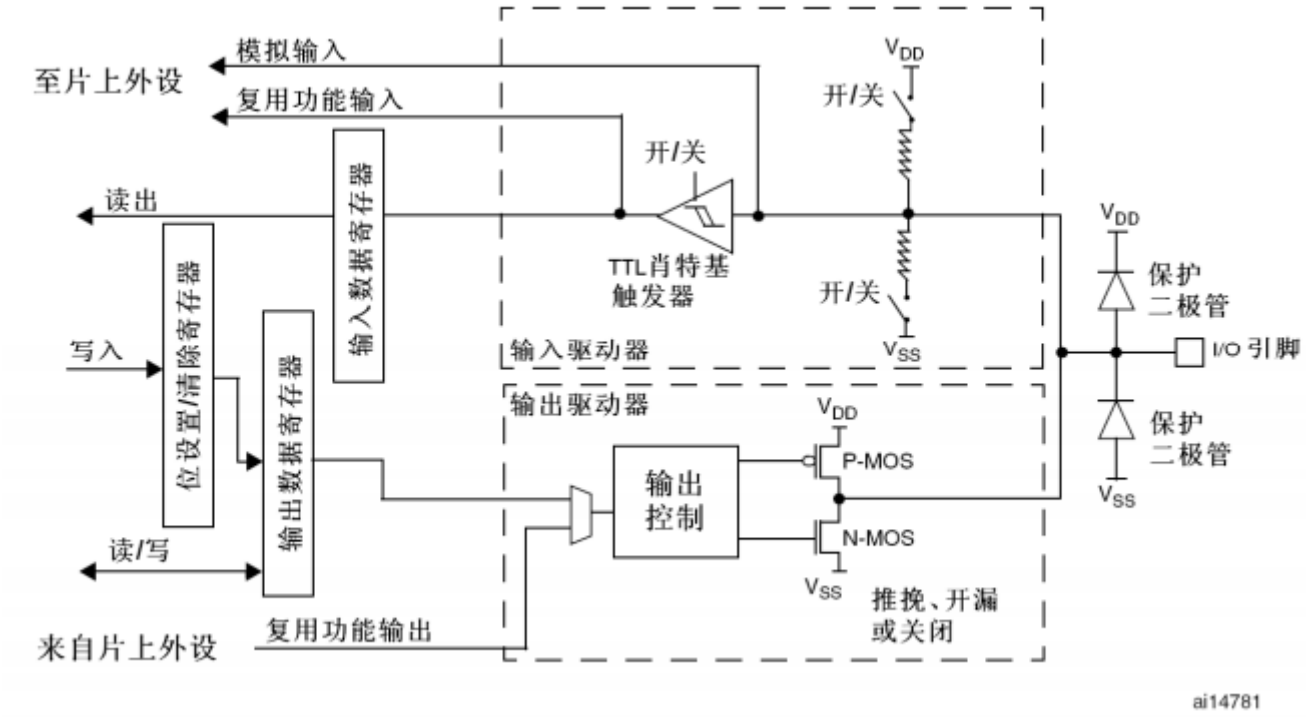
通用外设

GPIO

GPIO简介

GPIO，全称 通用输入输出（General Purpose Input/Output）。通过GPIO我们得以将STM32与外部设备的连接以实现各种功能。

工作原理及工作模式



GPIO硬件框图

1. 上下拉电阻

   通过控制上下拉电阻改变该引脚电压的默认电平

   ```
   上拉(Pull-up): 高电平
   下拉(Pull-down): 低电平
   浮空(No pull-up and no pull-down): 不定电平
   ```

   这里需注意,由于上下拉电阻的供电电压由芯片内部提供，因此输出的电流很小，所以上下拉三种模式一般用于处于输入状态的引脚，而需要输出较大电流时还是需要外部上拉。

7.4.4 GPIO 端口上拉/下拉寄存器 (GPIOx_PUPDR) (x = A..I/)

GPIO port pull-up/pull-down register

偏移地址：0x0C

复位值：

- 0x6400 0000（端口 A）
- 0x0000 0100（端口 B）
- 0x0000 0000（其它端口）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y:2y+1 **PUPDRy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入，用于配置 I/O 上拉或下拉。

00：无上拉或下拉

01：上拉

10：下拉

11：保留

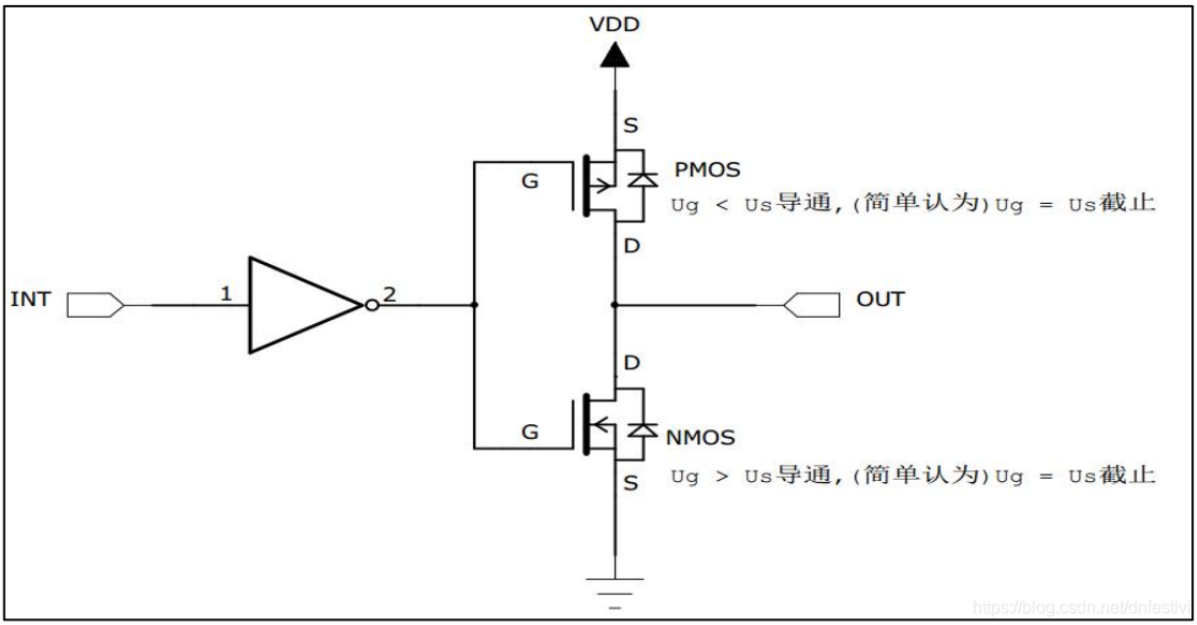
https://blog.csdn.net/dnfestiv

上拉/下拉寄存器GPIOx_PUPDR

2. MOS管

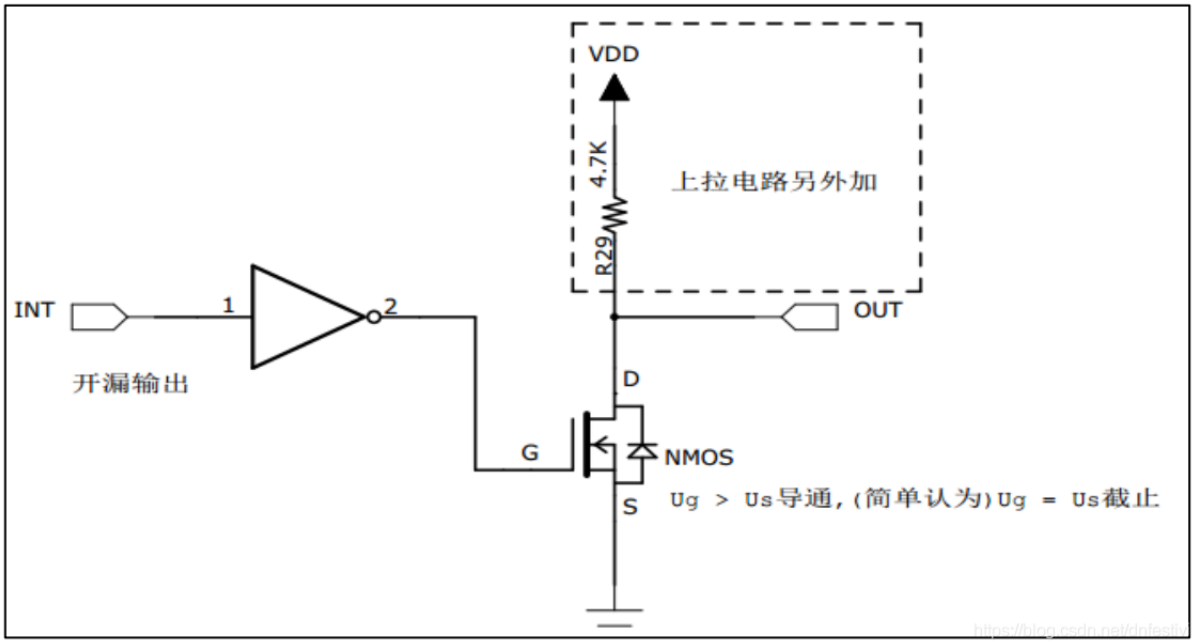
通过控制MOS管的通断改变该引脚的输出模式

- 推挽输出(GPIO_Mode_Out_PP)



通过配置输出寄存器控制在INT端输入高电平时，上方的P-MOS导通，下方的N-MOS关闭，对外输出高电平；而输入低电平时，经过反向后，N-MOS导通，P-MOS关闭，对外输出低电平，当引脚高低电平切换时，两个管子轮流导通，P管负责灌电流，N管负责拉电流，其负载能力和开关速度都比普通的方式有很大的提升。

- 开漏输出



这一模式下，上方PMOS不工作，控制INT端输出低电平，NMOS导通，输出接地；而输出高电平，NMOS关闭引脚处于高阻状态，此时电路具有"线与"特性，即只有当多个高阻态开漏输出的引脚接在一起，且外部上拉高电平时，引脚才会输出高电平，否则引脚输出低电平。另外，开漏输出还有可以应用在电平不匹配的情况，比如外部上拉电源5V。

7.4.6 GPIO 端口输出数据寄存器 (GPIOx_ODR) (x = A..I)

GPIO port output data register

偏移地址：0x14

复位值：0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:16 保留，必须保持复位值。

位 15:0 **ODRy[15:0]**：端口输出数据 (Port output data) (y = 0..15)
这些位可通过软件读取和写入。
注意：对于原子置位/复位，通过写入 GPIOx_BSRR 寄存器，可分别对 ODR 位进行置位和复位 (x = A..I)。

输出数据寄存器GPIOx_ODR

置位/复位寄存器GPIOx_BSRR

对于输出数据寄存器，我们又通过修改置位/复位寄存器修改输出数据寄存器的值。

3. 施密特触发器

   施密特触发器也称肖特基触发器，

4. 复用功能输出

   STM32的其他片上外设对GPIO引脚进行控制，此时GPIO引脚用作该外设的一部分，作第二用途使用。例如，我们在使用UART串口通讯时，就会将GPIO引脚复用为UART，并通过配置UART的寄存器控制该引脚。

5. 复用功能输入

   与复用功能输出类似

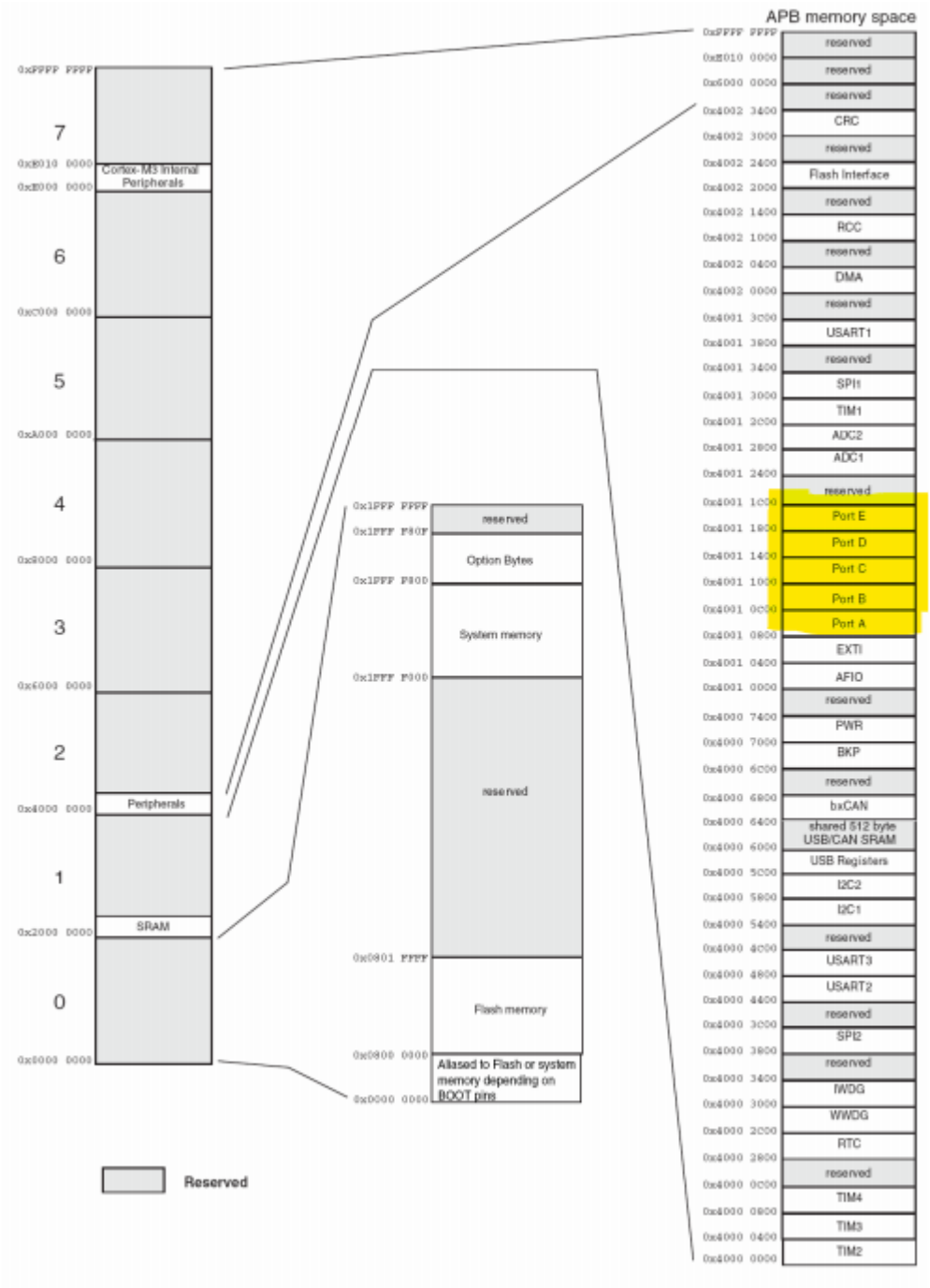
6. 模拟输入输出

   该引脚用作模拟输入功能时，输入的信号不经过施密特触发器（一种电压比较器），即输入到片上的信号为模拟信号，我们常用于ADC采集信号；而用于模拟输出功能时，输出的信号不经过双MOS管结构，直接将模拟信号输出至引脚。值得注意的是，引脚作为模拟输入输出时，引脚的上下拉电阻失效。

7. 中断输入

实际操作

1. 寄存器操作



stm32存储器映像

GPIO外设，修改端口高低控制寄存器GPIOx_CRH和GPIOx_CRL可以配置每个GPIO的工作模式和速度，每四位控制一个IO，CRL控制端口的低8位，CRH控制端口的高8位

端口配置低寄存器GPIOx_CRL

端口配置高寄存器GPIOx_CRH

2. 标准库操作

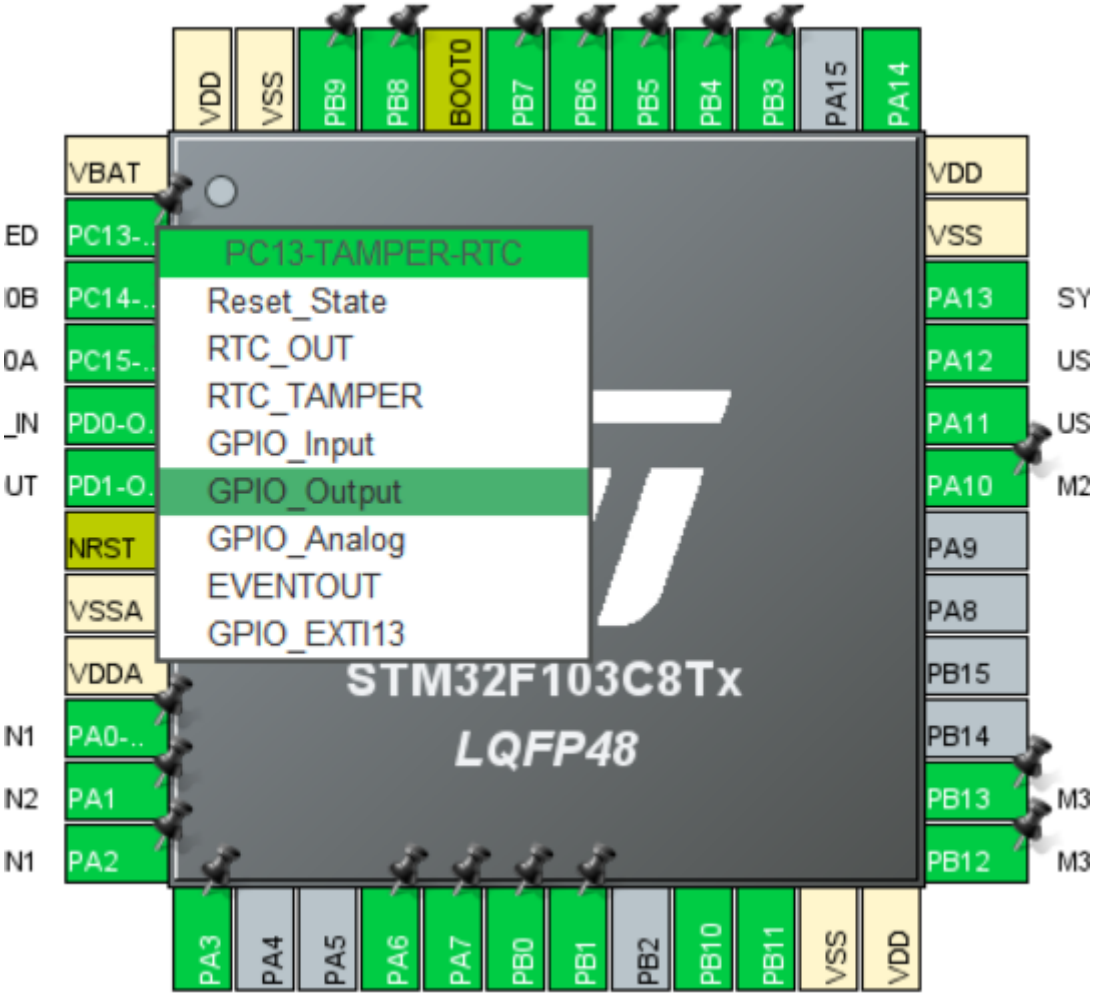
```c
/* GPIO初始化结构体参数 stm32f1x_gpio.h */
typedef enum
{
  GPIO_Speed_10MHz = 1,
  GPIO_Speed_2MHz,
  GPIO_Speed_50MHz
}GPIOSpeed_TypeDef;

typedef enum
{ GPIO_Mode_AIN = 0x0,                //模拟输入
  GPIO_Mode_IN_FLOATING = 0x04,       //浮空输入
  GPIO_Mode_IPD = 0x28,               //下拉输入
  GPIO_Mode_IPU = 0x48,               //上拉输入
  GPIO_Mode_Out_OD = 0x14,            //开漏输出
  GPIO_Mode_Out_PP = 0x10,            //推挽输出
  GPIO_Mode_AF_OD = 0x1C,             //复用开漏输出
  GPIO_Mode_AF_PP = 0x18              //复用推挽输出
}GPIOMode_TypeDef;

typedef struct
{
  uint16_t GPIO_Pin;                  //结构体元素1: GPIO_Pin
  GPIOSpeed_TypeDef GPIO_Speed;       //结构体元素3: GPIO_Speed
  GPIOMode_TypeDef GPIO_Mode;         //结构体元素2: GPIO_Mode
  }GPIO_InitTypeDef;
```

```c
/* 初始化GPIO引脚 */
GPIO_InitTypeDef  GPIO_InitStructure;                    //开启GPIO初始化的结构
体

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);    //使能APB2下的PORTA端
口时钟

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13;                //PC13端口配置
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;         //配置为推挽输出
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;        //设置IO口速度为50MHz
GPIO_Init(GPIOC, &GPIO_InitStructure);                   //结构体根据设定参数初
始化GPIOC13
```

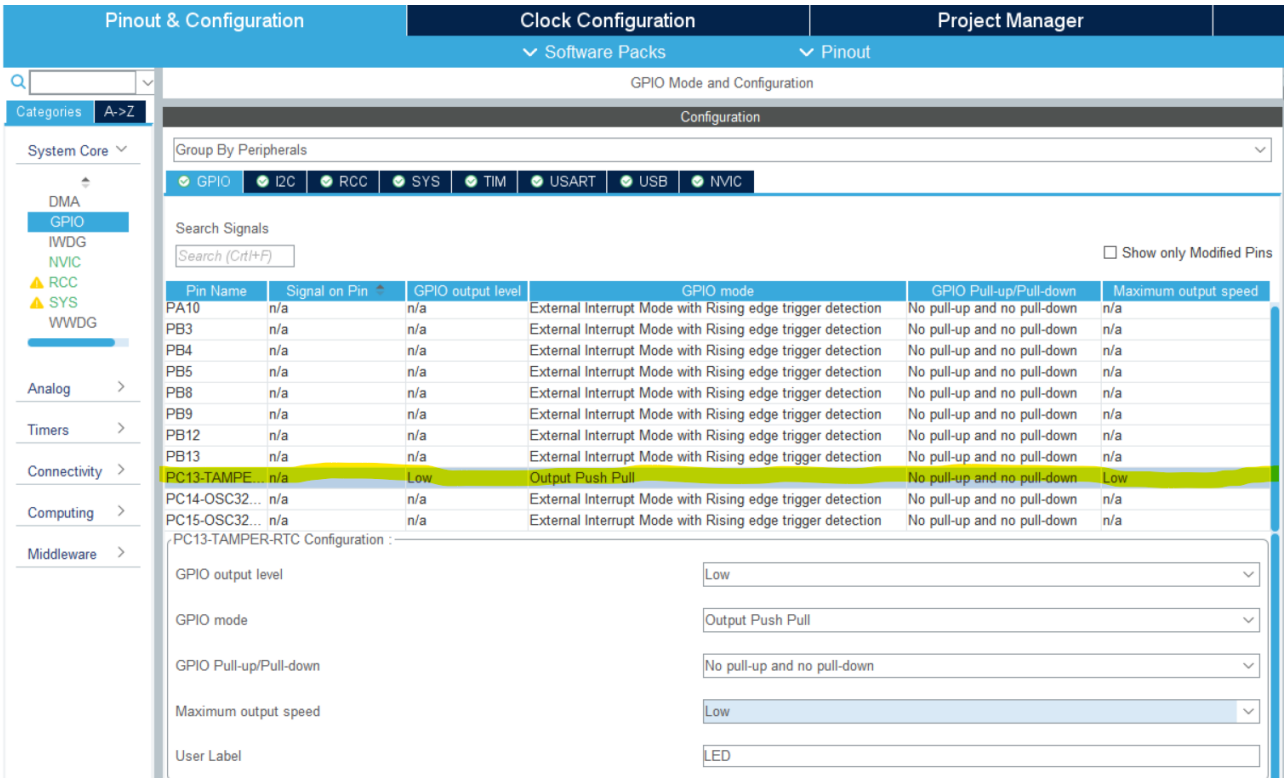
```c
/* GPIO操作 */
GPIO_SetBits(GPIOC,GPIO_Pin_13);                          //PC13 输出高
GPIO_ResetBits(GPIOC,GPIO_Pin_13);                        //PC13 输出高
```

3. HAL库操作

- Cube端配置


VDD
VSS
PB9
PB8
BOOT0
PB7
PB6
PB5
PB4
PB3
PA15
PA14
VBAT
PC13-..
PC14-..
PC15-..
PD0-O..
PD1-O..
NRST
VSSA
VDDA
PA0-..
PA1
PA2
PC13-TAMPER-RTC
Reset_State
RTC_OUT
RTC_TAMPER
GPIO_Input
GPIO_Output
GPIO_Analog
EVENTOUT
GPIO_EXTI13
STM32F103C8Tx
LQFP48
VDD
VSS
PA13
PA12
PA11
PA10
PA9
PA8
PB15
PB14
PB13
PB12
PA3
PA4
PA5
PA6
PA7
PB0
PB1
PB2
PB10
PB11
VSS
VDD

引脚开启


Pinout & Configuration
Clock Configuration
Project Manager
Software Packs
Pinout
GPIO Mode and Configuration
Categories
A->Z
System Core
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
Computing
Middleware
Configuration
Group By Peripherals
GPIO
I2C
RCC
SYS
TIM
USART
USB
NVIC
Search Signals
Search (Crtl+F)
Show only Modified Pins
Pin Name | Signal on Pin | GPIO output level | GPIO mode | GPIO Pull-up/Pull-down | Maximum output speed
PA10 | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PB3 | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PB4 | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PB5 | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PB8 | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PB9 | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PB12 | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PB13 | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PC13-TAMPE... | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low
PC14-OSC32... | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PC15-OSC32... | n/a | n/a | External Interrupt Mode with Rising edge trigger detection | No pull-up and no pull-down | n/a
PC13-TAMPER-RTC Configuration :
GPIO output level: Low
GPIO mode: Output Push Pull
GPIO Pull-up/Pull-down: No pull-up and no pull-down
Maximum output speed: Low
User Label: LED

引脚配置

```
output level //引脚输出电平
mode //引脚输出模式（开漏/推挽）
Pull-up/Pull-down //拉高/拉低
output speed //IO口速度
```

NVIC&EXTI

NVIC简介及EXTI概念引入

NVIC全称是Nested vectoredinterrupt controller，即嵌套向量中断控制器。

- 什么是“中断"？什么是中断服务函数？

  我们可以将中断从字面意思上理解，就是A程序被另外一个事件打断不再进行而去执行B程序。

  以饭馆吃饭为例，使用中断就是饭做好了，服务员会为你端上来，然后你开始吃饭（程序B），这种情况下服务员端上来之前你爱干啥就干啥（程序A）；但如果不使用中断，你需要一次一次去问服务员饭做好了没有，这期间你没办法去做其他事情。

  ```
  伪代码如下:
  /*不使用中断*/
  while(1){
       (询问服务员饭做好了没)
      if (饭做好了){
           (吃饭)
      }
  }
  /*使用中断*/
  [设置中断] (如果服务员把饭端上来了就进入中断服务函数)

  while{
       (干别的事)
  }

   (中断服务函数)
  ```

  那么理解了中断之后，我们再研究NVIC，即将不同的中断进行嵌套，并且与向量一样，有大小和方向，我们可以理解为配置每个中断的优先级。

- 中断的类型

  在微控制器中，我们将中断分为两种：芯片内核产生的中断，称为异常，也称内核中断；而芯片片上外设产生的中断称为外部中断，比如GPIO引脚电平变化、定时器溢出等。

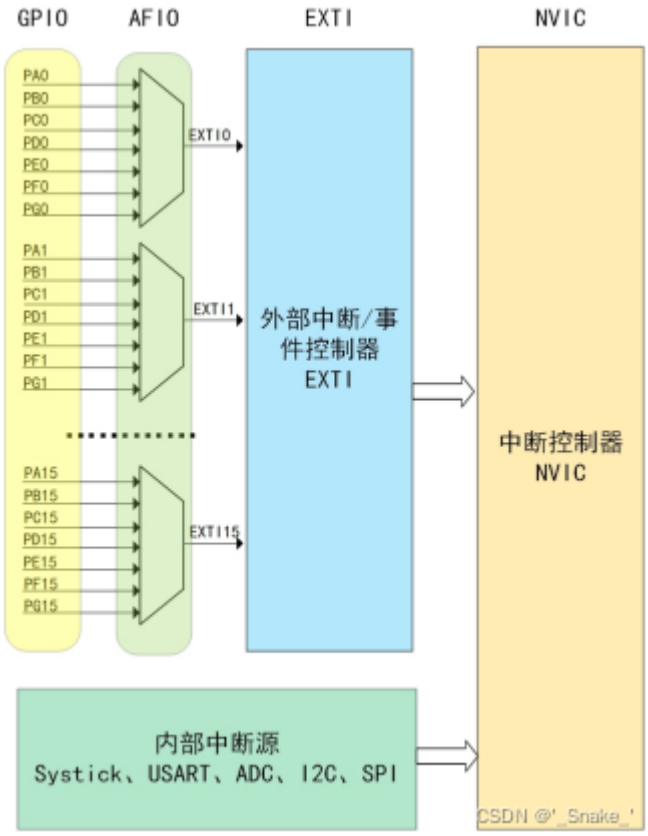
  NVIC与Cortex-M内核有着密不可分的联系，然而基于内核发挥作用的STM32片上外设不需要用上Cortex-M内核高达256种的中断类型。如STM32F1的小、中、大容量产品就仅仅有10个内核中断和60个外部中断，如图为STM32F1的10个内核中断。在这里我们可以理解为优先级数值越小，该中断的优先级越高，也即当多个中断发生时系统需先处理优先级高的中断。

位置	优先级	优先级类型	名称	说明	地址
	-	-	-	保留	0x0000_0000
	-3	固定	Reset	复位	0x0000_0004
	-2	固定	NMI	不可屏蔽中断 RCC时钟安全系统(CSS)联接到NMI向量	0x0000_0008
	-1	固定	硬件失效(HardFault)	所有类型的失效	0x0000_000C
	0	可设置	存储管理(MemManage)	存储器管理	0x0000_0010
	1	可设置	总线错误(BusFault)	预取指失败，存储器访问失败	0x0000_0014
	2	可设置	错误应用(UsageFault)	未定义的指令或非法状态	0x0000_0018
	-	-	-	保留	0x0000_001C ~0x0000_002B
	3	可设置	SVCall	通过SWI指令的系统服务调用	0x0000_002C
	4	可设置	调试监控(DebugMonitor)	调试监控器	0x0000_0030
	-	-	-	保留	0x0000_0034
	5	可设置	PendSV	可挂起的系统服务	0x0000_0038
	6	可设置	SysTick	系统嘀嗒定时器	0x0000_003C

STM32F10xxx产品(小容量、中容量和大容量)的内核中断向量表

对于内核中断，从图中我们可以看到，除了前三个异常以外，剩下的异常都可以直接通过NVIC进行控制（NVIC的控制原理参考《Cortex-M3内核编程手册》）

对于外部中断，我们又分为两类：一类其中断源来自外部引脚GPIO或复用的外部引脚AFIO，我们用外部中断/事件控制器(EXTI)进行控制；另一类为中断源来自不挂载在GPIO下的引脚。



外部中断类型 注：此处的内部中断源包括了可编程的内核中断systick和非GPIO中断，其中的systick不属于外部中断，不要被图片误导

EXTI工作原理

综上述，我们知道了EXTI，全称External interrupt/event controller，即外部中断/事件控制器。

- 那么NVIC和EXTI到底有什么作用，又有什么不一样呢？

  作为STM32的两大中断管理器。首先，他们都有共同的功能：中断使能和挂起位查询，但他们管辖的范围不同；其次，NVIC还有一重要功能————优先级管理。

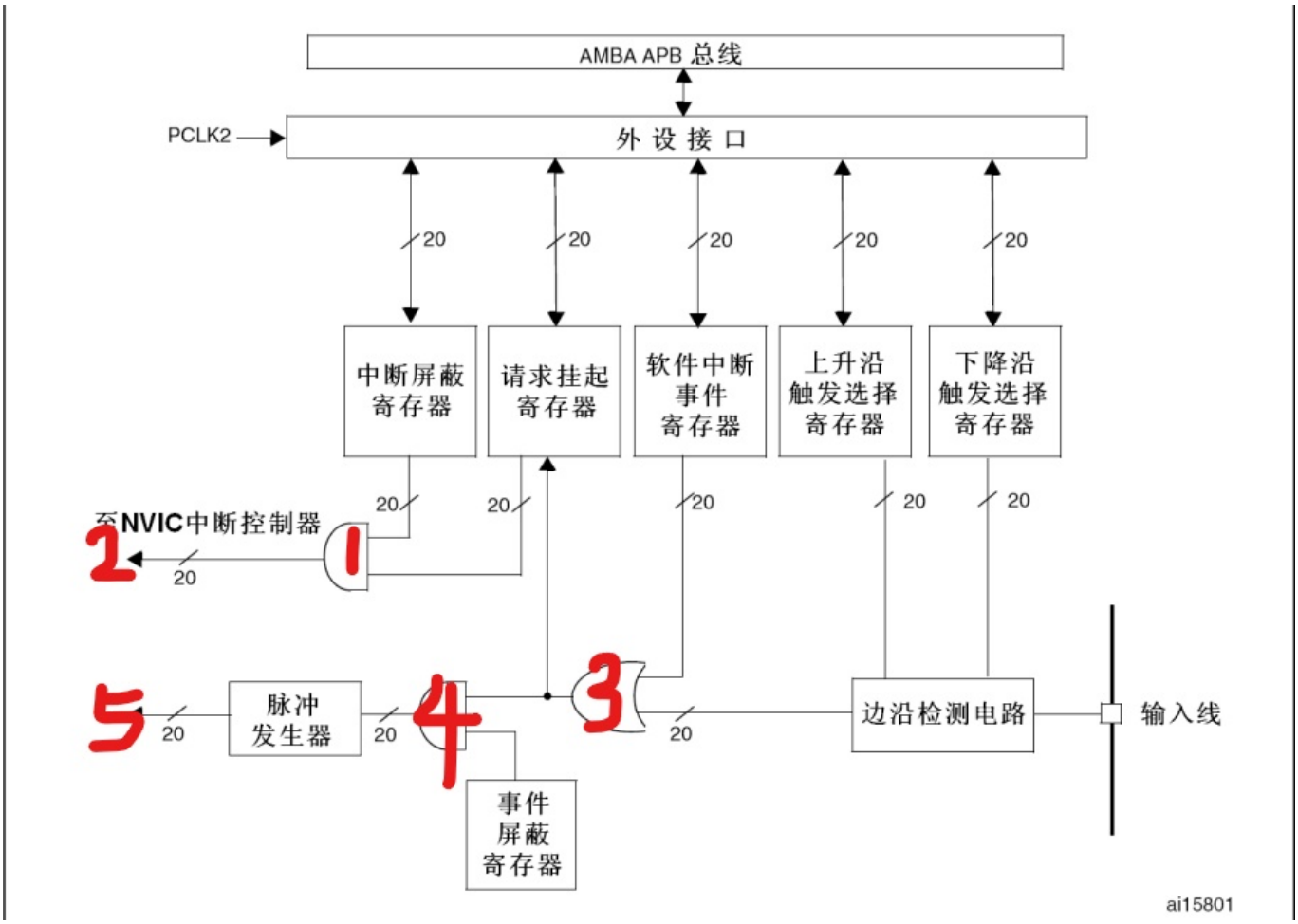
  我们可以举一个不恰当的例子来描述内核、NVIC、EXTI、外设之间的关系：大臣相当于外设，奏折相当于外设的中断/事件，太监相当于NVIC，皇帝相当于内核。大臣写了个奏折通过太监上报给皇帝进行处理。

- 什么是中断使能和挂起位查询？

  继续沿用那个“不恰当的例子”，EXTI使能就是大臣写了奏折，NVIC使能就是太监上报奏折给了皇上。而如果大臣意识到问题（中断信号）后却没有写奏折，或太监把奏折扣留了，则称为把中断挂起。

  在芯片内部，我们使能多个中断，如果中断发生时，正在处理同级或高优先级异常，或者被掩蔽，则中断不能立即得到响应，此时中断被挂起。挂起意味着等待而不是舍去，当优先级高的或者同级先发生的中断完成后，被挂起的中断才会执行。中断的挂起状态可以通过“中断设置挂起寄存器(SETPEND)”和“中断挂起清除寄存器(CLRPEND)”来读取，还可以手动挂起中断。

下面我们研究这一控制器是如何工作的。



STM32F1的EXTI硬件框图

EXTI具有两大功能：产生中断与产生事件。

- 中断和事件的区别？

  如之前提及的，产生中断的目的是把输入信号输入到NVIC中断控制器中，进一步执行中断服务函数，实现功能，这是软件级别的。而产生事件的目的是传输一个脉冲信号给其他外设使用，这是电路级别的信号传输，属于硬件级别的，比如可以给定时器TIM或者ADC等使用。

我们可以将最终传至NVIC的中断类型分为两种：

1. 输入信号配置为上升沿或下降沿触发

   当信号进入边沿检测电路，该电路由上升沿触发选择寄存器(EXTI_RTSR)和下降沿触发选择寄存器(EXTI_FTSR)配置该电路由上升沿还是下降沿触发中断标志位置1，继而电平信号和软件事件寄存器共同作为输入，进入3的“或门”电路，此时软件中断事件寄存器(EXTI_SWIER)不做配置置0，中断信号进入请求挂起寄存器(EXTI_PR)，若没有同级或高优先级中断，或者被掩蔽，则继续进入1的“与门”电路；若不操作中断屏蔽寄存器屏蔽特定中断，则置位为1，4处“与门”电路导通，中断电平信号输出到NVIC。

2. 输入线设定为事件

   边缘检测电路前同上述一致，但进入3的“或门”电路，通过配置软件中断事件寄存器(EXTI_SWIER)，将中断信号设置为某事件，若此事件发生，则电平信号继续进入4的“与门”电路；若不操作事件屏蔽寄存器屏蔽特定事件，则置位为1，4处“与门”电路导通，事件电平信号控制脉冲发生器产生脉冲至片上其他外设。

综上，我们可以看出边沿检测电路的作用是用于检测信号的上升沿和下降沿，3处“或门”电路适用于检测事件发生与否，4处“与门”电路用于屏蔽特定事件，1处“与门”电路用于屏蔽特定中断。

实际操作

无论是哪一种操作，我们首先都要明确两个概念：优先级和优先级分组

1. 优先级：抢占优先级和子优先级

   Cortex-M3将其利用NVIC中的中断优先级寄存器(NVIC_IPRx)将优先级分为0-255共8位，数值越低，优先级越高。但多数芯片不采用全部位数，比如STM32F103只用了高四位，这四位优先级又被分为抢占优先级和子优先级（响应优先级）。对于多个中断，先比较抢占优先级，若一致再比较子优先级，若优先级都相同，则比较硬件中断编号。

2. 优先级分组

   根据项目对不同抢占优先级和子优先级的不同需求，标准库又建立了优先级分组，由内核外设SCB的应用程序中断和复位控制寄存器(AIRCR)的PRIGROUP[10:8]位决定。比如STM32F103将4位的优先级分为16bits：

   NVIC_PriorityGroup_0：0bit抢占优先级，16bits子优先级；

   NVIC_PriorityGroup_1：2bits抢占优先级，8bits子优先级；

   NVIC_PriorityGroup_2：4bits抢占优先级，4bits子优先级；

   NVIC_PriorityGroup_3：8bits抢占优先级，2bits子优先级；

   NVIC_PriorityGroup_4：16bits抢占优先级，0bits子优先级；

- NVIC

  1. 寄存器操作

  2. 标准库操作

```c
/*NVIC优先级分组配置 misc.h*/
void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup);
//优先级分组参数
#define NVIC_PriorityGroup_0         ((uint32_t)0x700)
#define NVIC_PriorityGroup_1         ((uint32_t)0x600)
#define NVIC_PriorityGroup_2         ((uint32_t)0x500)
#define NVIC_PriorityGroup_3         ((uint32_t)0x400)
#define NVIC_PriorityGroup_4         ((uint32_t)0x300)

/* NVIC初始化结构体参数 misc.h */
typedef struct{
    uint8_t NVIC_IRQChannel;                    //NVIC中断源
    uint8_t NVIC_IRQChannelPreemptionPriority;  //NVIC抢占优先级
    uint8_t NVIC_IRQChannelSubPriority;         //NVIC子优先级
    FunctionalState NVIC_IRQChannelCmd;         //NVIC使能与否
} NVIC_InitTypeDef;
/*NVIC_IRQChannel中断源成员 stm32f10x.h*/
```

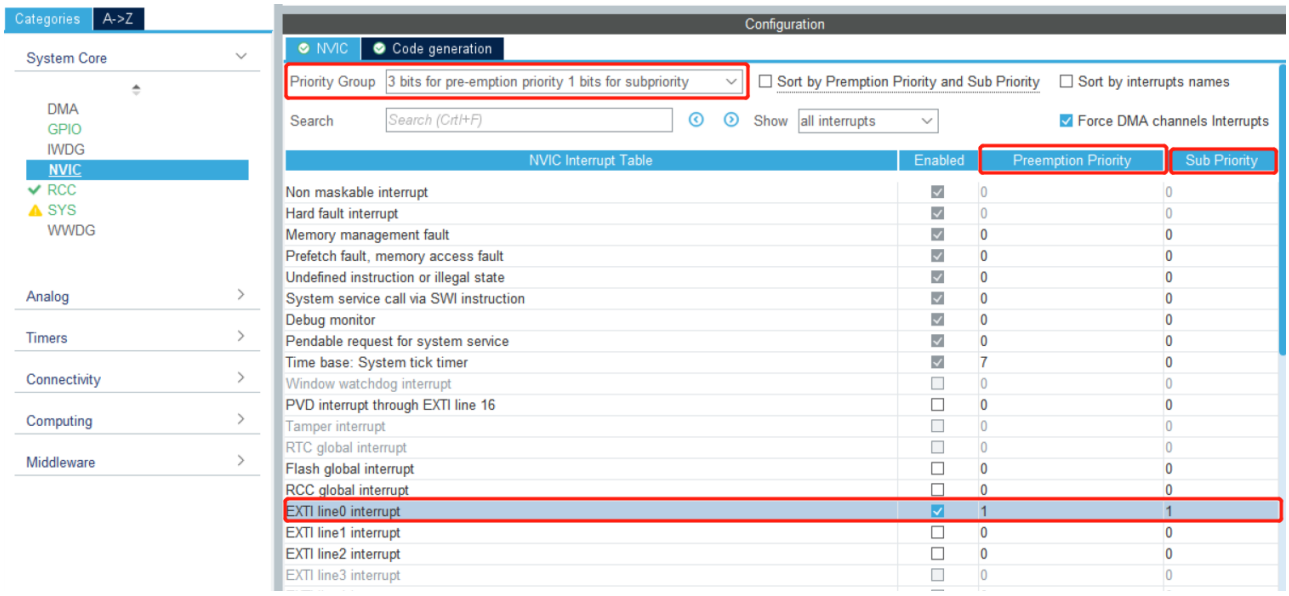
```
typedef enum IRQn
{
    /*内核异常*/
    NonMaskableInt_IRQn         = -14,
    MemoryManagement_IRQn       = -12,
    BusFault_IRQn               = -11,
    UsageFault_IRQn             = -10,
    SVCall_IRQn                 = -5,
    DebugMonitor_IRQn           = -4,
    PendSV_IRQn                 = -2,
    SysTick_IRQn                = -1,

    /*外部中断（部分）*/
    WWDG_IRQn                   = 0,
    PVD_IRQn                    = 1,
    //...
}IRQn_Type;
/*使能参数 stm32f10x.h*/
typedef enum {DISABLE = 0, ENABLE = !DISABLE} FunctionalState;
```

```
/* NVIC初始化操作 */
NVIC_InitTypeDef NVIC_InitStructure;                        //开启NVIC初始化结构体

NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);             //设置NVIC优先级分组为Group_1

NVIC_InitStructure.NVIC_IRQChannel = EXTI0_IRQn;            //配置中断源为EXTI0
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;   //设置其抢占优先级为1
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;          //设置其子优先级为1
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;             //使能中断通道

NVIC_Init(&NVIC_InitStructure);                             //初始化NVIC
```

```
/*其他NVIC操作 core_cm3.h*/
NVIC_EnableIRQ(EXTI0_IRQn);
NVIC_DisableIRQ(EXTI0_IRQn);
NVIC_SetPendingIRQ(EXTI0_IRQn);
NVIC_SetPriority(EXTI0_IRQn,  (1<<__NVIC_PRIO_BITS) - 1));
/*(1<<__NVIC_PRIO_BITS)-1)这个参数即抢占先优先级值，
其中__NVIC_PRIO_BITS是stm32.h中的宏定义，库函数默认为4，表示用4位表示占先优先级，
因为m3内核只有4位用来表示占先优先级和响应优先级，
那么响应优先级就剩下0位了，也就是没有响应优先级之分。
按照上式计算若NVIC_PRIO_BITS为4则占先优先级为15，
即最低优先级值依次可类推，
```

```
即 (1<<__NVIC_PRIO_BITS)-1)表示的是可用的最低优先级 (1<<4为16, -1为15) ,
将后面那个“1”改为其他值即可改变此模块抢占先优先级*/
```

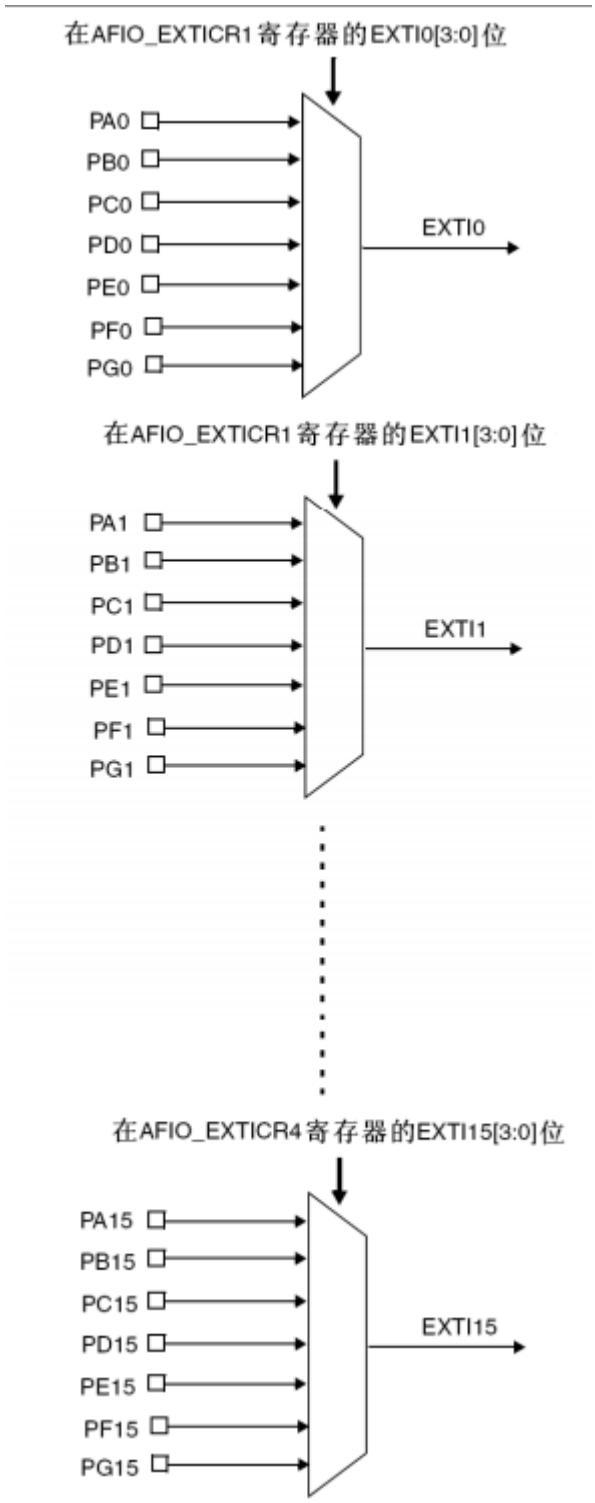
3. HAL库操作




```
PriorityGroup        //优先级分组
PreemptionPriority   //抢占优先级
SubPriority          //子优先级
```

- EXTI

1. 寄存器操作



STM32F1系例通用IO外部中断源映像

```
#define EXTI_Line0       ((uint32_t)0x00001)
#define EXTI_Line1       ((uint32_t)0x00002)
#define EXTI_Line2       ((uint32_t)0x00004)
#define EXTI_Line3       ((uint32_t)0x00008)
#define EXTI_Line4       ((uint32_t)0x00010)
//...
```

2. 标准库操作

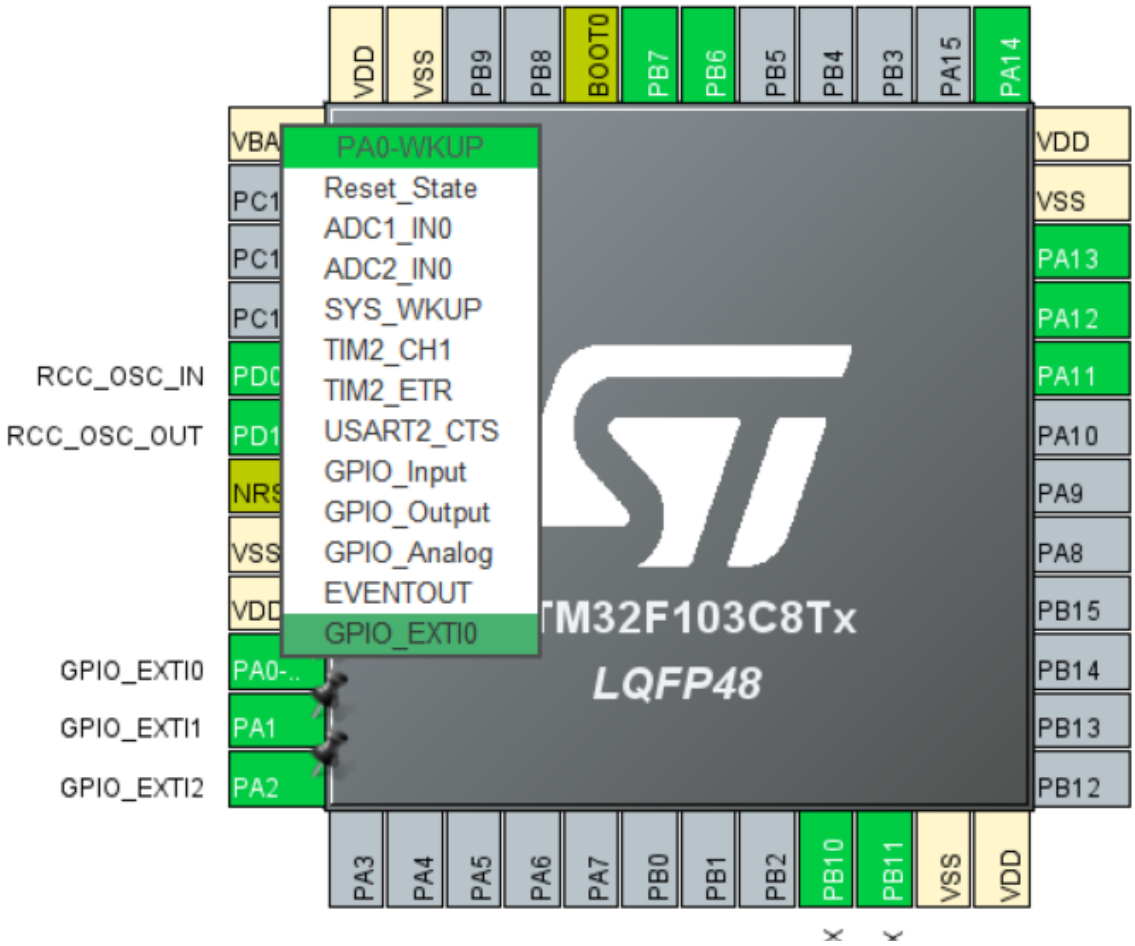
```
/*GPIO中断信号源选择 stm32f10x_gpio.h*/
//信号源选择函数
void GPIO_EXTILineConfig(uint8_t GPIO_PortSource, uint8_t
GPIO_PinSource);

//GPIO_PortSource
#define GPIO_PortSourceGPIOA       ((uint8_t)0x00)
#define GPIO_PortSourceGPIOB       ((uint8_t)0x01)
#define GPIO_PortSourceGPIOC       ((uint8_t)0x02)
//...

//GPIO_PinSource
#define GPIO_PinSource0            ((uint8_t)0x00)
#define GPIO_PinSource1            ((uint8_t)0x01)
#define GPIO_PinSource2            ((uint8_t)0x02)
#define GPIO_PinSource3            ((uint8_t)0x03)
//...


/*EXTI初始化结构体 stm32f10x_exti.h*/
typedef struct
{
    uint32_t EXTI_Line;                  //中断源
    EXTIMode_TypeDef EXTI_Mode;          //EXTI模式
    EXTITrigger_TypeDef EXTI_Trigger;    //EXTI触发类型
    FunctionalState EXTI_LineCmd;        //EXTI中断使能
}EXTI_InitTypeDef;

typedef enum
{
    EXTI_Mode_Interrupt = 0x00,      //中断
    EXTI_Mode_Event = 0x04           //事件
}EXTIMode_TypeDef;
typedef enum
{
    EXTI_Trigger_Rising = 0x08,          //上升沿触发
    EXTI_Trigger_Falling = 0x0C,         //上升沿触发
    EXTI_Trigger_Rising_Falling = 0x10   //上升沿触发
}EXTITrigger_TypeDef;

/*stm32f1高容量的中断服务函数 startup_stm32f10x_hd.s*/
__Vectors   DCD     RCC_IRQHandler
            DCD     EXTI0_IRQHandler
            DCD     EXTI1_IRQHandler
            DCD     EXTI2_IRQHandler
            DCD     EXTI3_IRQHandler
            DCD     EXTI4_IRQHandler
            DCD     DMA1_Channel1_IRQHandler
/*DCD是一条数据定义伪指令，用于分配一片连续的字存储单元并用指定的数据初始化。
当发生中断（异常）时，该异常被Cortex-M3内核接受，对应的异常Handler就会执行。
而这个响应过程都是硬件来完成的，
为了决定Handler的入口地址，Cortex-M3使用了“向量表查表机制”。
startup_stm32f10x_md.s启动文件已经为每个中断服务函数赋予了该地址，
```

```
所以在异常发生后，CPU进入异常模式，
同时程序计数器PC自动指向异常入口地址，也就是中断服务函数，
进而执行中断服务函数中的应用，执行完之后再回到主函数继续执行。*/
```

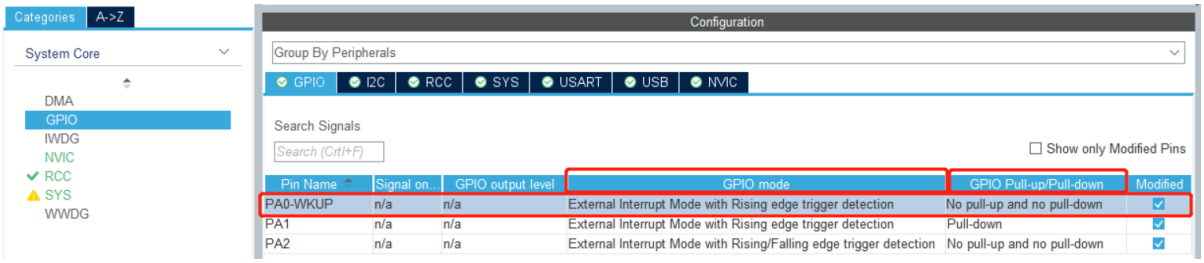
```
/*外部中断初始化*/
/*开启需要的初始化结构体*/
NVIC_InitTypeDef NVIC_InitStructure;
GPIO_InitTypeDef GPIO_InitStruct;
EXTI_InitTypeDef EXTI_InitStruct;

/*开启GPIO时钟*/
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);

/*NVIC初始化*/
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
NVIC_InitStructure.NVIC_IRQChannel = EXTI0_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStructure);

/*GPIO初始化*/
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
GPIO_Init(GPIOC, &GPIO_InitStructure);

/*EXTI初始化*/
GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource0); //输入信号源
为PA0
EXTI_InitStruct.EXTI_Line = EXTI_Line0;                     //中断源为
EXTI0
EXTI_InitStruct.EXTI_Mode = EXTI_Mode_Interrupt;            //设置为中断
触发
EXTI_InitStruct.EXTI_Trigger = EXTI_Trigger_Rising;         //设置为上升
沿触发
EXTI_InitStruct.EXTI_LineCmd = ENABLE;                      //使能EXTI中
断
EXTI_Init(&EXTI_InitStruct);                                //初始化EXTI
```

```
/*EXTI中断服务函数*/
void EXTI0_IRQHandler(void){
    //判断产生EXTI0的中断与否
    if (EXTI_GetITStatus(EXTI_Line0) != RESET){

        //中断服务函数应用

        EXTI_ClearITPendingBit(EXTI_Line0); //清除EXTI0的中断标志位
    }
}
```

3. HAL库操作



引脚配置



中断模式/触发模式配置

```
GPIO_Mode //中断/事件触发;上升沿下降沿触发
```

```c
/*中断回调函数*/
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    if(GPIO_Pin == GPIO_PIN_0){
        if(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0)==0){
            //中断回调函数应用
        }
    }
```

```
}
/*这里有两点需要注意:
第一，HAL中已经把本该是我们要去清除的标志位给清除了，
也就是说，我们在使用STM32CubeMX开发的过程中，
使用的任何中断都不需要去关心标志位的问题;
第二，此时中断的优先级高于systick，
而HAL_Delay()是由systick提供的时钟，
因此如果要在中断回调函数中使用此函数，
需使该中断的优先级低于systick的优先级*/
```

DMA

总线外设

UART/USART

UART简介